

Signals Protocol

Arnold Almeida
arnold@lighthouse.cx

James MacWhyte
james@lighthouse.cx

November 28, 2025

Abstract

Signals is a board-based on-chain coordination primitive where members lock an ERC-20 for time to express intensity of support on concrete initiatives. Each lock mints a transferable ERC-721 that represents the right to redeem the underlying after expiry or earlier upon initiative acceptance. Boards are immutable configurations at deployment time: lock interval, maximum lock duration (in intervals), decay curve and parameters, acceptance criteria (owner/anyone gating plus fixed and/or percentage thresholds), eligibility requirements for proposers and supporters, inactivity timeout, and optional post-acceptance release timelock. The protocol integrates optional reward mechanisms: a multi-board **IncentivesPool** that distributes board-wide participation rewards via time-bucketed weighting, and a per-initiative **Bounties** contract backed by an allow-listed **TokenRegistry**. Boards are deployed via a minimal-proxy factory and expose a simple lifecycle: propose, support (mint lock NFT), accept or expire, and redeem. This document preserves the system’s motivation and design goals while grounding the mechanisms in the shipped contracts and validated behaviors from tests and docs.

1 Introduction

Signals is an on-chain protocol that converts latent community sentiment into a real-time, ranked idea board of DAO initiatives. Inspired by conviction voting¹, commitment voting², and veToken systems such as Curve, Aragon, and Velodrome³, it lets members lock their existing governance tokens for a period of time to signal their support for what they think is most important. Locks mint transferable ERC-721 positions redeemable for the underlying after expiry or upon initiative success, aligning cost of participation with the intensity of belief.

A Signals instance (a *board*) is configured at deployment with immutable parameters: lock interval (seconds per epoch), maximum lock duration (in intervals), a decay curve and parameters, acceptance criteria, proposer and supporter eligibility requirements, inactivity timeout for expiring stale proposals, and an optional post-acceptance release timelock. These parameters are stored on-chain and enforced by the board contract.

Support is time-weighted via decay curves applied at the board’s lock interval. Acceptance can be gated by fixed and/or percentage-of-supply thresholds and access control (owner-only or anyone once thresholds are met). Optional incentive modules further align behavior: a multi-board **IncentivesPool** distributes rewards based on bucketed time weights, and a **Bounties** contract escrows ERC-20 rewards per initiative. Boards are created with a minimal-proxy factory to reduce deployment costs while keeping initialization flexible.

¹[1, 6].

²[3, 7].

³[5, 4, 2, 8].

1.1 Goals

- Deliver correctness and liveness guarantees under realistic network partitions.
- Keep the trust surface minimal and explicit (immutable board config, explicit thresholds).
- Make operational costs predictable across deployments via minimal proxies and efficient accounting.

1.2 Non-goals

Signals does not seek to replace binding on-chain execution frameworks nor to express complex multi-dimensional preferences. Endorsement-based extensions and advanced incentive schemas are scoped as optional modules and may evolve separately from the core.

2 Motivation

Token-based voting captures binary preferences on formal proposals but does little to surface intensity. After a decade of on-chain experimentation, DAOs still struggle to translate intent into timely, accountable action. Common pain points include:

- Endless discussion threads with unclear next steps.
- Committees that drift or concentrate power.
- Fragmented knowledge bases that deter newcomers.
- Social pressure that favors incumbents over fresh ideas.

By leveraging preference intensity through time-lock commitments, SIGNALS constructs a dynamic idea board where the most strongly supported initiatives rise naturally, keeping attention on what matters and when. Concretely, each position requires locking real tokens for time; supporters own an ERC-721 claim on those tokens and must wait for expiry (or success) to redeem, making opportunity cost explicit.

To counteract last-minute pile-ons and apathetic equilibria, the protocol optionally distributes participation rewards. A shared **IncentivesPool** funds supporters of accepted initiatives based on when they committed support (earlier commitments earn higher weight via time buckets), while a **Bounties** module lets contributors escrow targeted rewards per initiative. Both mechanisms aim to reward early, accurate signaling and raise the cost of low-conviction behavior without distorting the core lock-and-decay semantics.

3 Related Systems

Conviction Voting Holders stake without fixed locks; conviction builds the longer tokens stay committed and can be re-allocated at any time (which resets build-up). Proposals auto-execute once time-weighted conviction crosses an adaptive threshold tied to requested resources. It channels persistent conviction into binding, resource-allocating decisions.

Commitment Voting Voters lock tokens for a period to determine voting power. If their choice wins, tokens remain locked as committed; losing side unlocks immediately. Victory carries opportunity cost, so intensity matters. See [3, 7].

veToken Systems veCRV, Aragon OSx gauges, and Velodrome require duration-based locks to earn voting power. Lock length boosts influence across all votes but does not capture intensity on a specific initiative. See [5, 4, 8, 2].

Signals Signals borrows the idea of time-weighted commitment but applies it to a continuous idea board rather than a binary election. Unlike conviction voting, support decays deterministically at a board-defined interval and is bounded by a maximum lock duration. Unlike veToken systems, voting weight is scoped to a specific initiative via an ERC-721 lock position. Acceptance combines programmable thresholds with access control, and optional rewards are delivered by externalized modules to keep the core trust surface small.

4 Mechanism Design

4.1 Initiatives, not Proposals

SIGNALS works with *Initiatives*—ideas or actions seeking support. Initiatives live inside Boards (configuration sets), and DAOs can run multiple Boards with different parameters. Board configs are on-chain, so DAOs can govern them directly. A factory deploys new boards as minimal proxies of a reference implementation and calls `initialize` with the full configuration; successful deployments emit `BoardCreated`.

4.2 Board Configuration

A board is initialized with immutable parameters:

- **Underlying Token:** ERC-20 address used for locks (and used for eligibility checks).
- **Lock Interval:** Time granularity for lock and decay calculations (e.g., 1 day).
- **Max Lock Intervals:** Upper bound on lock duration (expressed in intervals).
- **Decay Curve:** `linear` or `exponential`, with parameters.
- **Acceptance Criteria:** `anyoneCanAccept`, `ownerMustFollowThreshold`, fixed threshold, and optional percentage-of-supply threshold. The effective threshold is $\max(\text{fixed}, \text{percentage} \times \text{totalSupply})$.
- **Eligibility:** Proposer and supporter requirements: `None`, `MinBalance`, or `MinBalanceAndDuration` (uses ERC20Votes checkpoints). Optional `minLockAmount`.
- **Inactivity Timeout:** Time after last activity when an initiative can be expired by the owner.
- **Release Timelock:** Optional delay before early redemption after acceptance.
- **Board Open/Closed:** `boardOpenAt` and `boardClosedAt` gate propose/support. Owner can also *close* or *cancel* a board (cancellation unlocks all locks for redemption).

Metadata for the board (and for initiatives) includes a title, body, and up to five attachments with non-empty URIs.

4.3 Token-Locking and ERC-721 Positions

Participants lock the board’s ERC-20 for a chosen duration (in intervals) to express commitment. Each lock mints a transferable ERC-721 position (one token per lock). The token name derives from the underlying ERC-20 name and the symbol is prefixed with **sx**. Positions are burned on redemption when the underlying is returned. A zero-duration lock carries nominal weight and is redeemable immediately; longer durations increase initial weight but are subject to decay.

4.4 Lifecycle

- **Propose:** `proposeInitiative` or `proposeInitiativeWithLock`. The latter both proposes and adds initial support. Propose/support require the board to be open and the caller to meet proposer/supporter requirements.
- **Support:** `supportInitiative` transfers underlying into the board, mints an ERC-721 lock, and updates initiative activity. Support can only be added while the initiative is **Proposed**.
- **Accept:** `acceptInitiative` moves **Proposed** $\rightarrow$ **Accepted**. If `anyoneCanAccept=false`, only the owner can call. If `ownerMustFollowThreshold=true` or a non-owner calls (when allowed), the initiative must meet the acceptance threshold at call time (sum of decayed weights).
- **Expire:** `expireInitiative` moves **Proposed** $\rightarrow$ **Expired** after the inactivity timeout; callable by the owner only.
- **Redeem:** `redeemLock` or `redeemLocksForInitiative`. A lock is redeemable if: the board is cancelled, the initiative is expired, the lock’s own duration has elapsed, or the initiative is accepted and the release timelock (if any) has passed. Redemption burns the lock NFT and transfers the underlying back. If incentives are configured, reward claims are processed atomically during redemption.

4.5 Calculating Support

Time is measured in discrete intervals (the board’s lock interval). Let a lock be created at time t_0 with amount T and duration D (in intervals). At time t , the number of elapsed intervals is $\lfloor (t - t_0)/\text{interval} \rfloor$. Weight contributions follow the chosen decay curve and are bounded below by the nominal amount:

$$\text{Weight}(t) = \max(\text{Curve}(T, D, \text{elapsed}), T).$$

For linear decay with rate $k \in [0, 1]$ (WAD precision),

$$\text{Curve}_{\text{linear}}(T, D, i) = T \cdot D - \frac{T \cdot i \cdot k}{1e18}.$$

For exponential decay with multiplier $m \in [0, 1]$ (WAD),

$$\text{Curve}_{\text{exp}}(T, D, i) = (T \cdot D) \cdot m^i.$$

Support for initiative i is the sum over active, non-withdrawn locks:

$$\text{Support}_i(t) = \sum_{\ell \in \text{ActiveLocks}_i} \text{Weight}_\ell(t).$$

This design preserves a guarantee: even at maximum decay, a supporters weight never drops below their nominal lock amount T .

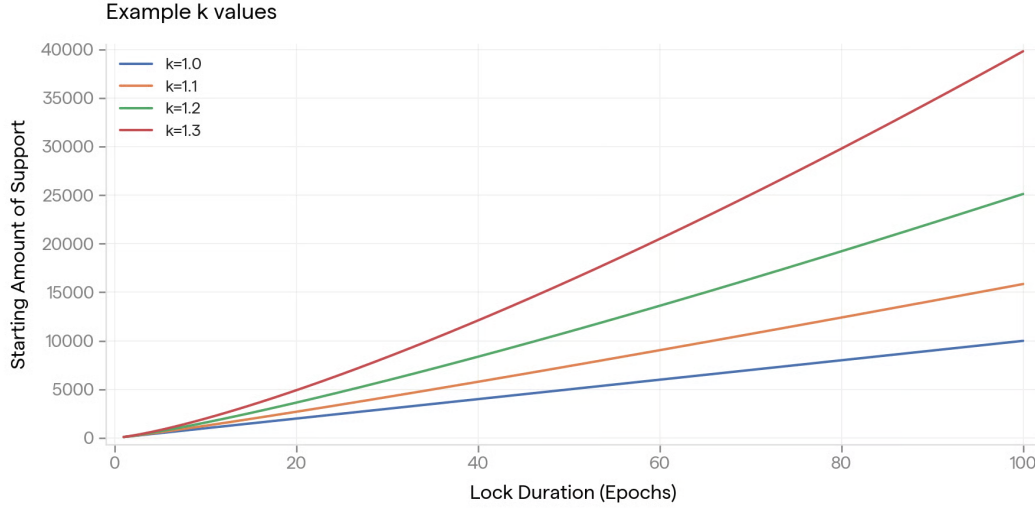


Figure 1: Decay curves (linear or exponential) applied at the board’s interval. Longer locks start with higher weight but decay toward the nominal amount.

4.6 Acceptance and Thresholds

Boards define acceptance criteria with two levers:

- **Access:** `anyoneCanAccept` gates whether non-owners can accept. Owners may optionally bypass thresholds when `ownerMustFollowThreshold=false`.
- **Thresholds:** a fixed threshold and an optional percentage-of-total-supply threshold; the effective threshold is the maximum of the two at decision time.

Once accepted, an initiative unlocks after the board-level release timelock (default 0), incentivizing supporters to back initiatives expected to succeed.

4.7 Inactivity and Expiration

Each initiative tracks *last activity*. If the initiative remains **Proposed** and no support is added for more than the board’s inactivity timeout, the owner can expire it. Expired initiatives stop accepting support and become redeemable immediately.

4.8 Optional Endorsements

If a DAO uses delegated voting, delegates may not own the tokens they wield. Boards could optionally require binary *Endorsements* tied to delegated voting power in addition to lockups. Caps on endorsements per Initiative encourage delegates to act early and visibly. This mechanism is not implemented in the current contracts and is left as a module for future work.

4.9 Factory Deployment

Boards are deployed via a minimal-proxy factory (EIP-1167 clones). The factory validates basic metadata (non-empty title, attachment limits) and emits `BoardCreated(owner, metadata)` upon success. This keeps per-board deployment gas low while preserving upgrade-independent logic in the reference implementation.

5 Paid Incentives in Signals

5.1 Motivation

DAOs often face low participation: most holders do not see sufficient upside to engage, and reputational risk pushes signaling to the last minute. Some DAOs spend \$1M+ per year on delegate programs. Signals inherits similar participation risks, so we add configurable incentives that reward early, accurate support and penalize weak or misaligned signaling.

5.2 Mechanism 1: General Participation Incentives (IncentivesPool)

A shared `IncentivesPool` can fund supporters of any initiative that gets accepted. The pool is independent from boards and can be shared across many (1:M). Its reward token is immutable and specified in the constructor; funds are added with `addFundsToPool`. Boards must be explicitly approved with two parameters: a per-board *max budget* and a *totalRewardPerInitiative* cap. Boards then link to the pool *before* they open.

When supporters add locks, the board records each lock's *incentive credit* in the pool using the lock's time-weighted contribution at creation. Credits are aggregated into a fixed number of time buckets (default: 24) with a starting interval (default: 1 hour). The board provides an *incentive curve* configuration (linear today, 2-24 parameters), and the pool converts the curve into per-bucket multipliers via interpolation and normalization. On redemption after acceptance, the board requests the pool to pay rewards for the redeeming locks: each lock gets a share proportional to its credited bucket and the curve's multipliers. The pool enforces the per-initiative cap and the board's remaining budget; if depleted, payouts simply become zero without blocking acceptance or redemption.

- **Setup:** deploy pool with `REWARD_TOKEN`, fund with `addFundsToPool`, approve boards with `approveBoard(board, boardBudget, totalRewardPerInitiative)`, link from the board before opening via `setIncentivesPool`.
- **Accounting:** time is bucketed and compressed (if buckets fill, they are reduced in-place to keep a bounded history).
- **Safety:** only approved boards can record credits and claim. Claims respect budgets and never revert for insufficient pool balance; rewards may be 0 if exhausted.

5.3 Mechanism 2: Sponsored Initiative Incentives (Bounties)

Anyone can add rewards to a specific initiative via the `Bounties` contract. Tokens must be on a `TokenRegistry` maintained by role-based access control. Bounties are stored with amount, optional expiration, and distribution *splits* configured by the owner (e.g., protocol/treasury/supporters). On acceptance, bounties are aggregated by token and split according to the active version of the configuration, crediting internal balances for recipients. Current limitations: refunding expired bounties is a planned enhancement and is not fully implemented yet.

5.4 Impact

Rewards amplify Signals core design: only positive support, opportunity cost via locks, and time-weighted influence that lets smaller holders compete. Incentives raise the cost of frivolous signaling and make early, accurate support economically meaningful, improving resilience against brigading or apathy-driven noise.

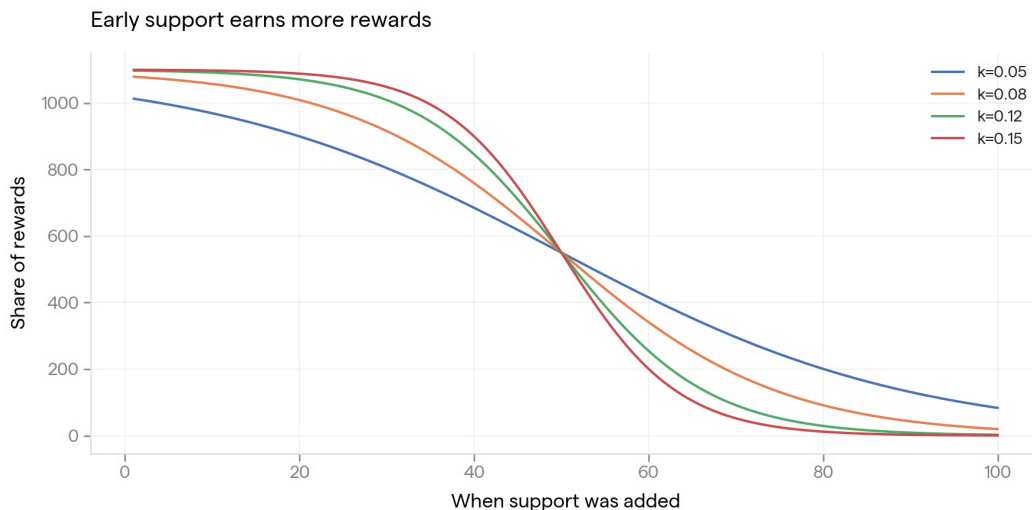


Figure 2: Example reward distribution emphasizing early supporters via bucket multipliers and shared upside.

5.5 Implementation Notes (from Tests)

End-to-end tests validate:

- Linking a board to a pool must happen before the board opens; otherwise calls revert.
- Reward distribution with a single supporter pays the `totalRewardPerInitiative` cap (subject to remaining budget).
- With multiple supporters at different times, earlier buckets receive higher multipliers (per linear config) and split rewards accordingly.
- Pool depletion is non-blocking: acceptance and redemption succeed; claims pay zero if budgets are exhausted.

6 What's Next

The Signals proof of concept was built during the Arbitrum Collabtech Hackathon (1st place) and continued in the Uniswap Hooks Incubator (prizes from Arbitrum and the Uniswap Foundation). Contracts and dashboard are open source.⁴

We aim to formalize and validate the protocol on:

- Capturing voter preference intensity and opportunity cost as sufficient risk.
- Locking as sybil resistance and its limits under heterogeneous token distributions.
- Gameable vs. productive patterns for endorsements and reputation (as an optional module).
- Inclusivity: how smaller voting blocks benefit from time-weighted designs and decay floors.
- Incentive design: robust bucket curves, interpolation strategies, and anti-gaming analysis.

⁴<https://github.com/OxLighthouse/signals>

- Bounties lifecycle: finalized refund semantics for expired bounties and contributor UX.
- Factory ergonomics and validation: richer config checks to prevent bricked boards.
- Gas and storage: empirical benchmarks for decay/claim paths and interval sizing.

Implementation work in progress:

- Additional decay and incentive curve variants with formal parameter bounds.
- Extended board state controls and analytics surfaces (weight snapshots, per-supporter queries).
- Documentation alignment and examples reflecting the constructor-based pool and board approval.

If interested in collaborating on a whitepaper or formal verification, please reach out.

References

- [1] 1Hive. Conviction voting. <https://github.com/1Hive/conviction-voting-cadcad/blob/master/README.md>, 2019. Accessed 2025-02-21.
- [2] Aragon. Introducing mode governance. <https://blog.aragon.org/introducing-mode-governance/>, 2024. Accessed 2025-02-21.
- [3] Jeremy Berg et al. Commitment voting: A mechanism for intensity of preference revelation and long-term commitment in blockchain governance. <http://dx.doi.org/10.2139/ssrn.3742435>, 2020. SSRN pre-print.
- [4] Curve DAO. Locked crv (vecrv) – overview. <https://resources.curve.fi/vecrv/overview/>, 2025. Accessed 2025-02-21.
- [5] Curve Finance. Understanding vecrv. <https://resources.curve.fi/reward-gauges/understanding-vecrv>, 2025. Accessed 2025-02-21.
- [6] Jeff Emmett. Conviction voting: A novel continuous decision making alternative to governance. <https://medium.com/giveth/conviction-voting-a-novel-continuous-decision-making-alternative-to-governance-aa746cfb9475>, 2019. Giveth blog.
- [7] Pouya Khezr et al. Voting with time commitment for decentralized governance: Bond voting as a sybil-resistant mechanism. <https://ssrn.com/abstract=4259599>, 2024. Management Science (forthcoming), SSRN pre-print.
- [8] Velodrome. Velodrome: Launch. part 1 – overview. <https://medium.com/@VelodromeFi/velodrome-launch-part-1-25b38120224a>, 2022. Accessed 2025-02-21.